



MODULE 10

Kotlin Programming Fundamentals

Enterprise Mobile Application Development Bootcamp

Phase 03 | Android Development with Kotlin

Module 10 Roadmap

Six blocks. By the end: comfortable reading and writing idiomatic Kotlin, coming from Swift.

1



Syntax &
Structure

2



Functions &
Lambdas

3



Null Safety

4



OOP &
Extension Fns

5



Coroutines

6



Collections,
Generics & Errors



Kotlin Syntax & Structure

- val vs. var
- Type Inference
- String Templates
- Control Flow: if/when



About Kotlin

- Kotlin was first released in 2016 by the JetBrains team in St. Petersburg
 - It is named after an island near St. Petersburg
- It runs on the JVM, so it is compatible with Java
 - It can also be compiled into JavaScript!
- It became very popular in 2017 when Google chose Kotlin as one of the formal development languages for Android development
- It is now bundled with Android Studio



Goals for Creating the Kotlin Language

- Compactness:
 - Developers wanted a concise language using less code to accomplish tasks
 - Comparisons show that code written in Kotlin is on average 30% shorter than the same code in Java
- Safety:
 - Kotlin's compiler detects many classes of errors that traditionally (in Java) were detected at run-time
 - e.g. Kotlin's compiler can prevent the dreaded NullPointerExceptions
- Interoperability
 - Kotlin works in perfect coordination with Java
 - Use Java standard libraries in Kotlin code
 - Extend Java standard libraries with Kotlin code
 - Mix Kotlin and Java code in the same project
- Less opinionated
 - You can write OOP, but you do not have to!
 - It also supports functional programming and lambda expressions



Kotlin Development Environments

- IntelliJ IDE
 - JetBrains' official IDE
 - Write code, compile, and execute within the IDE
- Kotlin Compiler
 - Separately downloadable
 - Write code in any editor, compile and run at the command line
- Android Studio
 - Another choice for IDE
 - Write code for Android applications, compile, and execute within the IDE
- Online editor
 - Options exist to enter and execute Kotlin code in a web browser



Resources

- Official website:
 - <https://kotlinlang.org/>
- Kotlin playground:
 - <https://try.kotlinlang.org/>
- Weekly Kotlin newsletter:
 - <http://www.kotlinweekly.net/>
- Kotlin for Android documentation
 - <http://developer.android.com/Kotlin/index.html>



val vs. var, and Type Inference

Swift

```
let accountName = "Everyday Checking"  
var balance = 4281.16  
balance -= 42.50
```

Kotlin

```
val accountName = "Everyday Checking"  
var balance = 4281.16  
balance -= 42.50
```

val → let. var → var. Both languages infer the type from the assigned value (String, Double here) without an explicit type annotation, and both let you write one explicitly when it aids clarity: `val balance: Double = 4281.16`.

NOTE: Kotlin is case-sensitive!



Primitive Data Types

- Kotlin data types are similar to Java's
 - Int
 - Long
 - Double
 - Float
 - Short
 - Byte
- There is NO implicit casting
 - Even for a widening conversion!

Code Bite

```
val x = 2          // inferred type
val y: Int         // declared type

val z: Int 2       // declared and inferred
                  // declaration is unnecessary

val w: Int = 4.6    // compile error!

var y: Double
y = x              // compile error!
```



Explicit Casting

- Every primitive data type contains methods for casting to any other type
 - toByte()
 - toInt()
 - toLong()
 - toDouble()
 - toFloat()
 - toShort()
 - toChar()

Code Bite

```
var x = 5
var y: Double

y = x.toDouble()    // compiles
```



String Templates

- Muscle-memory trap:
- Typing \() out of Swift habit inside a Kotlin string produces a literal backslash-paren in the output, not an error
- Which makes this an easy bug to miss.
- Watch for it in early exercises.

Code Bite

```
val name = "Checking"  
val balance = 227.38  
  
println("Account: $name")  
  
println("Double: ${balance * 2}")
```



Branching with if

- **if** is Kotlin's primary syntax for branching

Code Bite

```
if (balance < 100) {  
    print("Account balance is low.")  
}  
  
if (balance > 100) {  
    print("Account balance is good.")  
} else {  
    print("Account balance is low.")  
}
```



Branching with if

- **if** can be used as an expression, not just a statement
- This is Kotlin's version of a ternary operator

Code Bite

```
val status = if (balance < 0) "Overdrawn"  
              else "Current"
```



Control Flow: if and when

- If you chain together multiple **if else** tests together, comparing the same variable, **when** can provide a simpler syntax
- An entire block of code can be executed for a **when** branch

Code Bite

```
val feeDescription = when {  
    balance < 0 -> "Overdraft fee applies"  
    balance < 100 -> "Low balance warning"  
    else -> "No fees"  
}
```



Control Flow: if and when

- As with **if**, **when** can be used as an expression
- It can return a value

Code Bite

```
val accountType = when (typeCode) {  
    1, 2 -> "Checking"  
    3 -> "Savings"  
    else -> "Unknown"  
}
```



Block 1 - Kotlin Syntax & Structure

Skill Check ✓

- What are val and var each equivalent to in Swift?
- What's the Kotlin syntax for embedding an expression (not just a variable) inside a string template?
- Name one capability when has that a traditional switch statement doesn't.



Functions & Lambdas

- Function Syntax
- Default & Named Arguments
- Higher-Order Functions
- Trailing Lambda Syntax



Function Syntax

- Functions begin with the keyword **fun**
- That is followed by the function name
- And then the parameter list
 - Parenthesis are required even if there are no parameters
- The body of the function is enclosed in curly braces ({ })
- If the function returns a value, its return type is listed after the parameters and before the body of the function

Code Bite

```
fun sayHello() {  
    println("Hello Kotlin world!")  
}  
  
fun makeGreeting(name: String): String {  
    return "Hello, $name"  
}
```



Default Arguments

- Function parameters can have default values
 - After the parameter name and data type, assign a value
 - When called, if a value is provided for that parameter that value will be used
 - If no value is provided, the default value will be used
- This makes invoking the function more flexible

Code Bite

```
fun calculateFee(amount: Double,  
                 feeRate: Double = 0.03) {  
    return amount * feeRate  
}  
  
var fee = calculateFee(100.0)  
var specialFee = calculateFee(100.0, 0.05)
```



Named Arguments

- When calling a function, you can assign the parameter values to the parameter names.
 - Instead of passing raw values positionally
- This way, the order of the parameters does not matter
- It can make it easier to invoke a function with many parameters with default values and provide a non-default value for a parameter far down the list

Code Bite

```
fun calculateFee(amount: Double,  
                 feeRate: Double = 0.03) {  
    return amount * feeRate  
}  
  
var fee = calculateFee(amount = 100.0)  
var specialFee = calculateFee(  
    amount = 100.0,  
    feeRate = 0.05)
```



Single Expression Function

- In Kotlin, a **single-expression function** is a concise way to write a function whose body consists of only one expression
- Instead of writing curly braces and an explicit **return** keyword, you assign the expression directly to the function signature using an = symbol
 - The **return** is implied

Code Bite

```
fun calculateFee(amount: Double,  
                feeRate: Double = 0.03) =  
    amount * feeRate
```



Higher-Order Functions

- Functions can be passed to another function as a parameter value
- They also can be returned from a function
- When passing a function using its name, you need to use the function reference operator (::)

Code Bite

```
fun displayAccount(acc: Account) {  
    . . .  
}  
  
loadAccount(252, ::displayAccount)
```



Lambda Expression

- In Kotlin, a **lambda expression** is a short, unnamed block of code that can be treated as a value
- This means you can assign a lambda to a variable, pass it as an argument to a function, or return it from a function
- The data type specification can be inferred
 - In either direction!

Code Bite

```
val sum: (Int, Int) -> Int =  
    { x: Int, y: Int -> x + y }
```

```
val sum = { x: Int, y: Int -> x + y }
```

```
// OR
```

```
val sum: (Int, Int) -> Int =  
    { x, y -> x + y }
```



Single Parameter Shorthand for Lambda

- If a lambda expression has EXACTLY one parameter, you do not have to specify the parameter name
- Instead, you can use the keyword **it** to refer to the parameter

Code Bite

```
val square: (Int) -> Int = { x -> x * x }
```

```
// OR
```

```
val square: (Int) -> Int = { it * it }
```



Trailing Lambda Syntax

- When the last parameter of a function is itself a function, you can pass a lambda expression to it outside the parenthesis

Code Bite

```
loadAccount(242, { acct ->
    println("${acct.id} ${acct.balance}")
})
```

// OR

```
loadAccount(242) { acct ->
    println("${acct.id} ${acct.balance}")
}
```



Block 2 - Functions & Lambdas

Skill Check ✓

- What Swift feature is Kotlin's trailing lambda syntax directly equivalent to?
- What does it refer to inside a single-parameter lambda, and what's its Swift equivalent?
- How do you call a function while skipping some default-valued parameters?

